

Rewarding New Ideas in Reader Comments: Design Notes

A plain-language explanation of what I built, why I built it this way, how I judged success, what the results were, and what still doesn't work.

1. The problem in one paragraph

A news site publishes a short article, and readers post comments on it. Most comments repeat the same few reactions ("this will kill local shops", "great for kids"), and a few say something genuinely new. I wanted a system that looks at each new comment and gives it a **reward between 0 and 1**:

- **High** when the comment adds an idea nobody has raised yet, *and* it's actually about the article;
- **Low** when it repeats what others have already said;
- **Zero** when it's about something else entirely, however original it is.

The last point matters most. A comment about the local football team is "new" in the sense that nobody else mentioned football, but it shouldn't earn anything, because it isn't a response to the article.

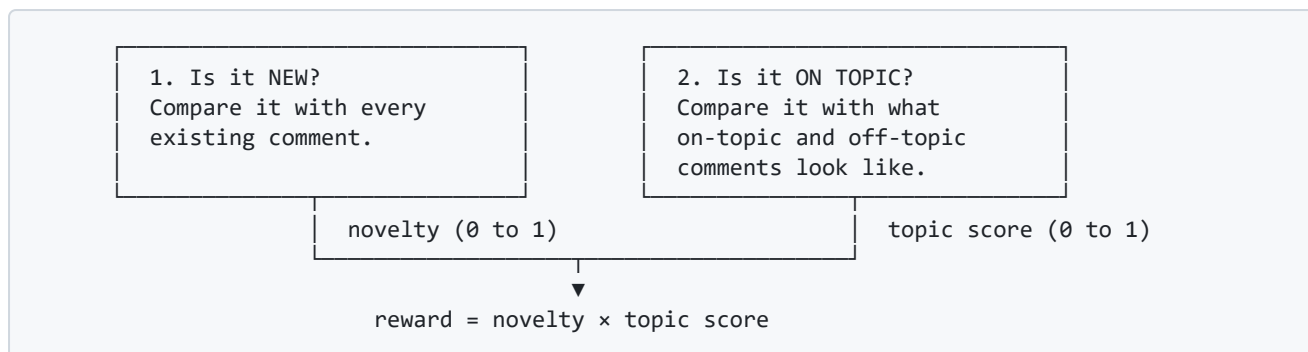
2. What I built, in simple terms

The setting

- **The article:** a 95-word local news story. The Riverton city council votes to tear down a downtown parking garage and build a park, paid for by a business tax and state grants, with a shuttle bus during construction.
- **A comment** has three parts, all written or chosen by the reader:
 - a **headline**,
 - a **body** (the comment itself),
 - a **stance**, picked from *support*, *oppose*, *mixed* or *undecided*.
- **The existing comments:** 50 example comments, written to look like a real comment section. There are many repeats of the obvious takes (parking, families, cost, heat, the shuttle) and only a few one-offs. I made that clustering deliberate: "new" only means something when most comments aren't.

How a comment gets its reward

Every new comment has to answer two questions:



Because the two numbers are multiplied, a comment has to do well on **both** to earn much. A brilliant idea that's off-topic gets 0, because 0 times anything is 0. A perfectly on-topic comment that just repeats others gets close to 0, because its novelty is close to 0.

How "new" is measured.

- **What the comment is about:** a small AI model (an "embedding model") turns each comment into a list of numbers that captures its meaning, an "embedding". Two comments that make the same point get similar numbers even if they use different words.
- **The words it uses:** the system also checks for shared distinctive words, like "stormwater" or "basements".
- **The judgement:** a comment that sits far from every existing comment is new.
- **Relative, not fixed:** "far" isn't a fixed number. The system first measures how far a *typical* existing comment is from the others, and then asks whether the new comment is further out than that. So a novelty of 0.5 means "about as new as a typical existing comment", and that stays true as comments are added.

How "on topic" is measured (the topic check).

- **Learning from examples:** the system learns what this article's topic looks like from two sets of examples. One set is the existing comments on the article. The other is 72 comments from the same town about *other* things: bus routes, the library, water bills, other parks, potholes, school news. These are deliberately tricky look-alikes, sometimes called "hard negatives".
- **The boundary:** it draws the line that best separates the two sets. A comment clearly on the "other things" side gets a topic score of 0, one clearly on the article's side gets 1, and one close to the line gets something in between.

Extra checks that can only lower the reward:

- A comment that is a near-copy of an existing one, or of the article itself, gets 0.
- A comment that isn't a real sentence (a list of keywords, one word repeated, a wall of emoji) gets 0.
- A comment that's too vague to be new ("I love this idea!") is scaled down.
- The stance can trim the reward by up to 10%: a comment taking the most common stance keeps 90% of it, and rarer stances keep more. Because it only scales the reward down, it can never turn a copy into a reward.

Cleaning up the text first. Before any of this, the text is tidied:

- invisible characters are removed, and look-alike letters from other alphabets (such as a Cyrillic "a") are swapped for the ordinary English letter, so a disguised copy is still a copy;
- common typos are fixed and texting shorthand is expanded, so bad spelling neither hides a repeat nor penalises a good idea;
- sentences that aren't in English are left out of the scoring, so they neither add to nor lower the reward.

Learning as it goes. When a comment is accepted, it joins the pool of existing comments, and the next person to make the same point gets a lower reward. That is what "new relative to other submissions" means. A comment is accepted only if it passes the topic check (even partly), isn't a near-copy of an existing comment or of the article, and contains real sentences. There is no minimum novelty, so a low-scoring rewording still joins. These rules stop spam and copies from shifting the system's picture of a typical comment, which is the yardstick for "new".

What you can run

The scorer comes with a command-line tool, a JSON web API and a small web page. On the page you type a comment, see its score and the reasons for it, and can choose to add it to the pool.

3. Why I built it this way

I made the main choices below by trying alternatives and measuring the results. Smaller settings (the 10% stance factor, the vagueness rule, the near-copy threshold) were tuned on this comment section and haven't been tested elsewhere.

Decision	What I tried first, and why it failed	What I chose
Compare meaning and words	Meaning alone missed rewordings: a reworded version of an idea already posted still scored 0.60, when it should have scored near 0. A bigger embedding model scored it even higher (0.81).	Blend meaning (60%) with shared distinctive words (40%). The reworded idea then dropped to about 0.1.
Judge "new" relative to the other comments	A fixed "far enough to count as new" number depends on the embedding model, so it would need re-tuning whenever the model changed.	Compare each comment with how spread out the existing comments are.
Check comments point by point	A comment listing every common opinion in one long sentence looked "new", scoring 0.81, because it didn't match any single comment.	Also score each clause separately; the list now scores 0.23.
Learn the topic from examples	Plain similarity to the article let a same-town football comment through. The next version asked whether a comment was closer to the article's topic than to general small talk; it still let local comments about <i>other</i> subjects through, with bus cuts and library hours scoring about 0.9 when they should have scored 0. Off-the-shelf AI models built to judge whether one text follows from another, or how well a passage matches a search, also put comments in the wrong order.	Learn the boundary from on-topic comments versus same-town off-topic comments.
Fix spelling before comparing	Misspelled words looked like new vocabulary, so a typo-filled repeat scored 0.86.	Correct common typos first; the same comment now scores 0.02.
Test on unseen data	My original tests were written by the same AI agent that built the system, so they shared its blind spots: they passed while the system failed on realistic comments.	Build a separate evaluation set of 200 comments and report results only on the part never used for tuning.

I followed two engineering principles throughout:

- **Measure before changing.** Every design change and speed-up started with a measurement and ended with a before-and-after number.
- **Keep it modular.** Each check is a separate, pluggable piece. Adding a new one, such as a moderation filter or an AI judge, doesn't require touching the rest.

4. How I defined success

The project brief sets its requirements; I turned each into a bar that a single comment either passes or fails, then measured what share of comments pass it. Because a novelty of 0.5 means "as new as a typical existing comment", I put the bar for a new idea at 0.6 (clearly newer than typical), for a repeat at 0.25 (clearly less new), and for off-topic at 0.01 (effectively zero).

Requirement	What counts as success
Reward truly new, relevant comments	Reward of 0.6 or more
Don't reward repeats	Reward of 0.25 or less for copies and rewordings of existing comments
Don't reward off-topic comments, even very original ones	Reward of 0.01 or less (effectively zero)
Stay relevant to the article without blocking genuine responses	At least 90% of the existing comments keep a full topic score when each is checked as if it were new
Scores on a 0-to-1 scale, relative to the other comments	Every score in [0, 1], and an idea's reward drops once someone has made it

Only the off-topic requirement has a target share: at least **85%** of unseen off-topic comments must score 0.01 or less, and a test enforces it. For new ideas and rewordings I set no target; I report the measured share, and the tests only stop it from falling about 10 points below today's value.

Three kinds of evidence:

1. **Automated tests (189 in total)** covering scoring, cheating attempts, unusual input, the maths and the server. The scoring tests check these bars on hand-picked cases, including 21 deliberate attempts to cheat the system.
2. **A separate evaluation set** of 200 comments, kept apart from the building work.
 - 140 were written by AI writers who saw only the article and the example comments, never the scoring code.
 - The other 60 are "red-team" comments (written to trick the system) from an independent review panel that had already tested it.
 - Each comment is labelled with what kind it is: new idea, rewording, vague or off-topic. A second AI labeller re-labelled every comment without seeing the first label, and only comments both agreed on were kept.

I split them in two:

- **dev (70):** the only part I tuned settings on;
- **test (130):** never used to tune any setting. All 60 red-team comments are here.

These are the numbers to trust most, because no setting was tuned on them. One caveat: the red-team comments include the very failures (bus cuts, library hours) that made me redesign the topic check, so the off-topic numbers are somewhat flattered.

3. **A hand check** with at least 10 human-written comments, not generated by AI. They scored as I expected. This is a spot check rather than a measured pass rate, so the numbers in section 5 come from the tests and the evaluation set.

5. Results

On the hand-picked cases

Kind of comment	Reward
Three genuinely new ideas (flood-proof park design, keeping the garage frame, a heritage walk)	0.84–0.85
Copies and rewordings of existing comments	0.00–0.04
Off-topic comments (sourdough baking, quantum computing, the high-school football team)	0.00
Six local comments about other town issues (bus route, library, water bills, snow plowing, polling place, school hours). Similar subjects appear in the training examples, so this shows the fix working where it was taught.	0.00 (previously 0.45–0.97)
Attempts to cheat: stuffing keywords, listing every opinion, repeating a word 40 times, padding with spam, disguised copies	0.00–0.23 (previously up to 0.81)

On the separate test set (130 comments never used for tuning)

What I measured	Comments	Before the new topic check	Now
Off-topic comments scoring 0.01 or less	43	33%	91%
Of these: same-town subjects the training examples covered	9	0%	89%
Of these: same-town subjects the training examples didn't cover	6	50%	100%
Of these: distant subjects	6	100%	100%
Tricky cases, such as an off-topic comment that mentions parking in passing	7	71%	86%
How well it ranks on-topic above off-topic (AUC)	–	0.79	0.91
New ideas scoring 0.6 or more	29	66%	55%
Rewordings kept at 0.25 or less	38	53%	50%
Vague comments kept at 0.25 or less	5	80%	80%
How well it ranks new ideas above the rest (AUC)	–	0.79	0.78
Rewarded until someone makes the point, then not	4 pairs	0%	0%

In plain words:

- **Off-topic comments:** the system now refuses to reward about 9 in 10 of them (39 of 43), including all 6 on subjects missing from its training examples. That sample is small, and similar subjects appeared in the part I tuned on.

- **New ideas:** it gives 0.6 or more to about half of them (55%), down from 66%, because the stricter topic check now holds back some genuine ideas (see Limitation 2).
- **Rewordings:** it keeps about half of those that use different words at 0.25 or less.
- **Rewards dropping once an idea is made:** on unseen pairs this never worked (0 of 4).

Scorecard

Success criterion	Status
Scores on a 0-to-1 scale	Met
Off-topic comments get nothing	Largely met: 39 of 43 unseen off-topic comments (91%) scored 0.01 or less, above the 85% target, including all 6 on subjects missing from the training examples (a small sample). The 4 that got through are close neighbours of the topic (another park's dog run and tennis courts, library funding, recycling) and scored 0.16–0.94.
Genuine comments aren't blocked	Mostly met: 94% of the existing comments keep a full topic score, and on-topic comments rank well above off-topic ones (AUC 0.91). But 2 of the 50 existing comments would be blocked if posted today, and the stricter check holds back some genuine new ideas.
Repeats don't get rewarded	Met on hand-picked cases: copies and rewordings that reuse words score 0.00–0.04. Partly met on unseen data: 19 of 38 rewordings in different words (50%) and 4 of 5 vague comments stayed at 0.25 or less.
New ideas get rewarded	Partly met: 16 of 29 unseen new ideas (55%) reached 0.6, down from 66%. The three hand-picked ideas score about 0.85. Of the 13 unseen ideas that fell short, 6 weren't judged new enough, and 7 were clearly new but held back, at least partly, by the topic check.
An idea's reward drops once someone has made it	Not met on unseen data: 0 of 4 test pairs passed (the idea must score 0.6 or more before it's posted, and its rewording 0.25 or less after). It works in a hand-picked case where the rewording shares key words: the first comment making the point scores 0.86, and a later rewording of it 0.06.

Speed and running cost

- **Speed,** for a comment section of about 50 comments on an ordinary laptop:
 - scoring a new comment takes about 15 thousandths of a second, and under 1 thousandth if the exact same text has been scored before;
 - adding an accepted comment and updating everything takes about 4 thousandths.
 - **Cost:** by default it runs fully offline, using a small, free embedding model on the same computer, so there's no per-comment fee. The system can also use Google's paid Gemini service instead. From token counts and Google's published price, I estimate that 1,000 new comments would cost about one to two US cents. That is an estimate, not a real bill, because the Google option hasn't been run with a real account.
-

6. Limitations: what still doesn't work well

1. **Rewordings in completely different words** are the biggest weakness. If someone restates an existing point with none of the same words, the system often thinks it's new. That is also why rewards don't reliably drop once an idea has been made (0 of 4 unseen pairs). The fix I'd make next is a second check that asks directly whether two comments make the same point, using a more careful AI comparison.
 2. **The strict topic check has a cost:**
 - it holds back genuine new ideas that sit near another subject. On the test set, 7 of the 13 new ideas that fell short were held back at least partly by the topic check, for example night-shift nurses who rely on the garage, or putting the comment form in Spanish. The stricter check caused 3 of these, which is why the share of new ideas reaching 0.6 fell from 66% to 55%. Falcons nesting on the garage is another case, from the tuning set;
 - a comment that is half off-topic chatter and half a good idea now earns nothing;
 - two short on-topic comments in the example set (a question about the shuttle, and a complaint about the vote) would be blocked if posted today.
 3. **The closest neighbours of the topic still slip through sometimes:** comments about *other* parks (a dog-run fence, cracked tennis courts), library funding and recycling.
 4. **It's built for one article.** The topic check learns from off-topic examples written for this article (I used 72), so a new article needs its own set. With fewer than 10, it falls back to a simpler check that is known to be weaker.
 5. **English only.** Non-English sentences are left out of the scoring, and a sentence that mixes English with another language earns nothing, even when it contains a good idea.
 6. **It judges newness, not quality or truth.** A new but rude or false comment is rewarded like any other new comment. The score is also shown instantly, so someone can keep rewording a comment until it scores well. A real deployment would put this behind moderation and rate limiting.
 7. **The Google option is untested.** It has never been run with a real account, and the hand-set settings (the 60/40 blend, the vagueness rule, the topic check's strictness) were tuned with the free local model only.
 8. **Built for a small comment section.** The speeds above are for about 50 comments. Each new comment is compared with every existing one, so tens of thousands of comments would need a faster search index.
-

7. Glossary

Term	Meaning here
Embedding	A list of numbers that captures what a piece of text means, so similar meanings get similar numbers.
Embedding model	The AI component that turns text into an embedding. By default the system uses a small, free one that runs on the same computer.
Novelty	How far a comment is from every existing comment, compared with how far apart the existing comments usually are.
Topic score	A number from 0 to 1 that multiplies the reward: 0 if the comment isn't about the article, 1 if it clearly is. The code calls it the "relevance gate".
Hard negatives	Off-topic examples that look deceptively similar to on-topic ones (same town, same tone), used to teach the topic check where the line is.
Red-team comments	Comments written deliberately to trick the system.
Dev / test split	The evaluation comments divided in two: settings are tuned only on <i>dev</i> , and results are reported on <i>test</i> , so the reported numbers aren't flattered by tuning.
AUC	A score from 0 to 1 for how well the system ranks one kind of comment above another: 0.5 is no better than guessing, 1.0 is perfect.